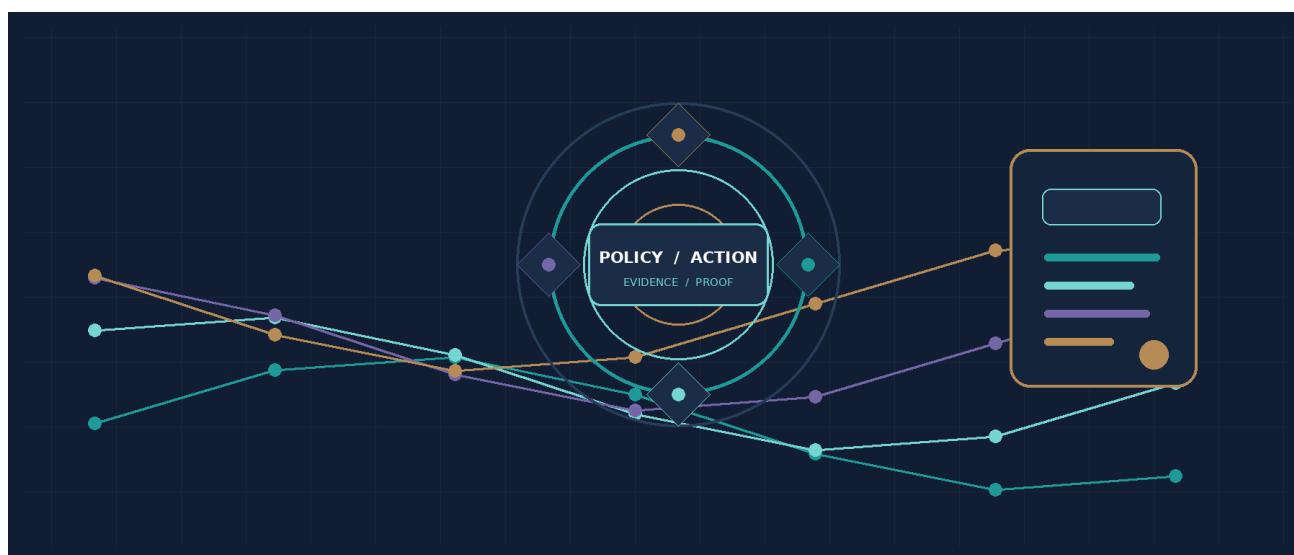


Governing AI at the point of action

Counterpoise, Verity and the Datafolio wallet

A technical white paper on executable policy control, data assurance and portable evidence



EDITION

Technical architecture overview

STATUS

Product concept / in development

OWNER

AIdentity • Datalogos Limited

DATE

September 2026

READ THIS PAPER / Purpose and central proposition

AI systems can now request data, call tools and initiate consequential work. The central design question is therefore not only what a model can generate, but which specific actions an organisation will permit, under what conditions, using which information, and how that decision can later be reconstructed.

This paper describes a proposed control architecture from AIdentity. Counterpoise coordinates policy evaluation and runtime authority across Governance, Privacy, Ethics and Security. Verity, its assurance capability, records the evidence and state changes connected to those decisions. Datafolio is the customer-controlled wallet concept for retaining and presenting selected records, credentials and proof receipts.

The architecture separates probabilistic reasoning from deterministic authority. AI can propose an action and provide context; bounded domain engines assess it; a deterministic Policy Decision Point (PDP) applies the organisation's rules; a named person approves where policy requires judgement; a short-lived signed authority is issued only when prerequisites are met; and a Policy Enforcement Point (PEP) checks that authority immediately before the side effect.

The aim is controlled delegation: let AI perform useful work within explicit boundaries, while preserving human accountability, least privilege, privacy and evidence. This is a design proposition, not a claim that a finished, certified production service is currently deployed.

AT A GLANCE / Product roles

Capability	Role in the design	What it contributes
AIdentity	Advisory	Strategy, operating model, governance design and implementation planning.
Counterpoise	Control plane	Coordinates domain assessments, deterministic decisions, approvals and bounded execution authority.
Verity	Assurance capability within Counterpoise	Records provenance, validation, signed manifests, commitments, state changes and assurance status.
Datafolio	Customer-controlled wallet concept	Holds or references records, credentials, permissions and proof receipts for scoped presentation.

Key boundary: Verity can help establish what state was assessed, what evidence was referenced, which rules and approvals were recorded, and whether a later state matches a commitment. Cryptography does not independently prove that the source information was true, lawful, complete or fit for purpose.

AIDENTITY / COUNTERPOISE + VERITY

Layered governance and assurance architecture

A conceptual view of how AI-enabled work is controlled, evidenced and presented as scoped proof.

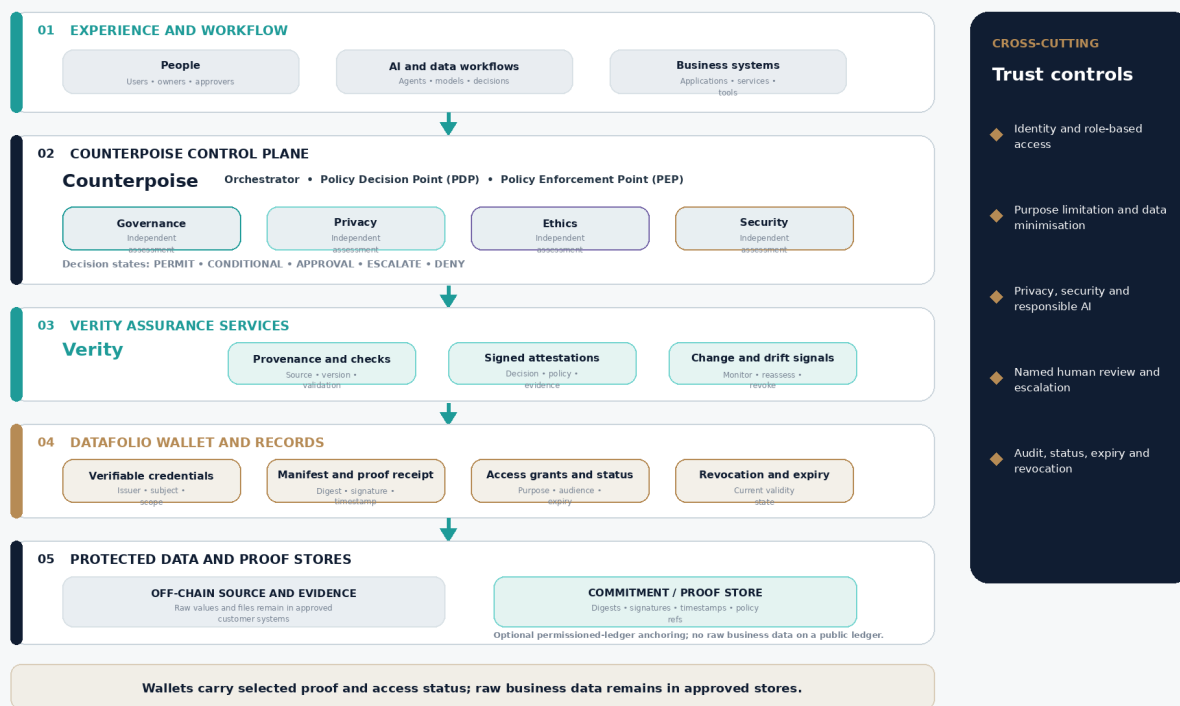


Figure 1. Layered architecture view. The five operating layers share cross-cutting trust controls; the full system, modular composition and data paths are detailed later.

01 / Why govern at the point of action

Many governance controls are designed as documents, reviews or periodic approvals. These establish intent, but they may not intervene when an AI agent is about to read a restricted record, send a message, change a system configuration or create a financial commitment. An agent's broad credentials are not a substitute for action-specific authority.

Counterpoise is designed around the moment where intent becomes a side effect. Each request is evaluated as a relationship among an actor, action, resource, purpose, destination and time window. The result is scoped: permission to perform one approved operation does not create blanket trust in an agent, model, user or connector.

This distinction matters because the risk context can change. A policy may be updated, consent withdrawn, a data asset altered, a model version replaced, an agent identity suspended, or a security incident declared. Previously granted authority or assurance may no longer be valid. The architecture therefore includes expiry, re-checks, change events and revocation as operating behaviours, rather than treating governance as a one-off pre-deployment gate.

DESIGN PRINCIPLES / The rules that shape the architecture

- Deterministic authority: only the deterministic PDP can issue the cross-domain execution decision; model output cannot grant itself permission.
- Action-level governance: bind each decision to a specific actor, operation, resource, purpose, destination and validity period.
- Independent control domains: Governance, Privacy, Ethics and Security return separate findings; a mandatory hard prohibition cannot be overridden by another domain.
- Enforcement at the side effect: a PEP validates the signed authority and its scope immediately before a protected action executes.
- Unknown is not safe: missing, stale or conflicting mandatory context leads to approval, escalation or denial under policy, never silent permission.
- Wallet first: customer records stay authoritative in customer-controlled systems; the platform uses references, claims and minimum necessary context by default.
- Evidence by design: material assessments, approvals, authority issuance, PEP checks and outcomes are provenance-linked and integrity-protected.
- Portable semantics: the same policy outcomes, schemas, reason codes and conformance tests apply across deployment profiles.

ARCHITECTURE BOUNDARY / What the platform does and does not own

Counterpoise coordinates control decisions and supplies bounded authority; it is not the system of record for customer business data. The Orchestrator coordinates work, resolves required context and manages state; it does not itself make the final cross-domain decision or execute the target action. Domain engines assess within their scope; they cannot bypass the PDP or PEP. Verity adds

evidence and assurance state; it does not certify the legal sufficiency of an organisation’s policy or the factual accuracy of source records.

02 / Counterpoise control plane

The logical control plane contains a trust edge, Orchestrator, Policy Administration Point (PAP), Policy Information Points (PIPs), bounded domain engines, deterministic PDP, human approval gate, authority service and PEP. An event and observability plane records lifecycle events and operational signals without logging customer content by default.

An inbound request is authenticated and schema-validated at the gateway. Tenant and actor identity are bound to authenticated claims; caller-supplied tenant headers are not trusted on their own. The Orchestrator resolves the minimum required facts from authoritative sources such as identity services, data catalogues, privacy registers, model registries and security tooling. Facts carry timestamps and provenance so policy can distinguish current evidence from stale or unavailable context.

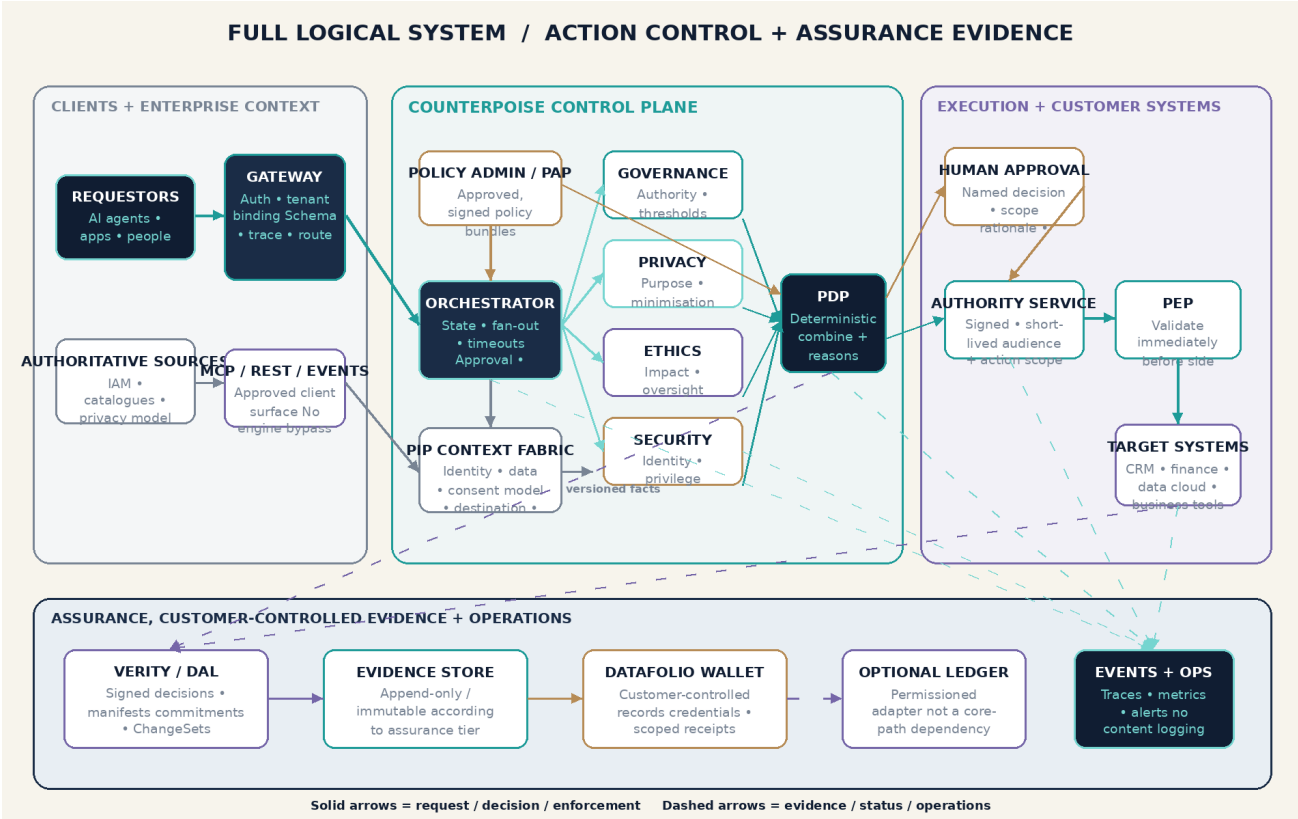


Figure 2. Full system view. Solid paths carry requests, decisions or enforcement; dashed paths carry evidence, status or operational events.

Component	Responsibility	Boundary
Trust edge	Authenticate clients; bind tenant and actor; validate request schema; assign trace IDs; rate-limit and route.	No policy decision authority.

Component	Responsibility	Boundary
Orchestrator	Create a decision transaction; call required PIPs and engines; manage timeouts, approvals and evidence coordination.	Cannot bypass the PDP or execute target actions.
PIPs and context fabric	Resolve current identity, purpose, resource, data, model, consent and security facts from authoritative systems.	Facts must be time-stamped and provenance-linked.
G / P / E / S engines	Assess the request under domain rules and return structured findings, reasons, obligations and evidence references.	Bounded assessment only; no execution authority.
Deterministic PDP	Apply the signed policy bundle and combining algorithm to the required domain results.	Sole cross-domain decision authority.
Human approval gate	Capture named approval, rationale, conditions and expiry when policy permits human judgement.	Cannot override non-overridable legal or security prohibitions.
Authority service and PEP	Mint short-lived scoped authority after prerequisites; validate it immediately before execution.	Fail closed on stale, invalid, out-of-scope or replayed authority.
Verity and evidence plane	Create signed decision records, manifests, commitments, receipts, change states and revocation information.	Integrity and provenance evidence; not source-truth proof.

INDEPENDENT FINDINGS / Why domain separation matters

A single blended “risk score” can hide conflicting reasons. Counterpoise instead retains each required domain result and its provenance. A privacy result might require masking; a security result might require multi-factor authentication; an ethics result might require human review; a governance result might restrict financial authority. The PDP combines those findings through a deterministic, versioned policy. This makes the reason for the final outcome inspectable and helps avoid one domain silently neutralising another domain’s restriction.

ENGINE INTERACTION MODEL / Shared context, separate accountability

The four domain engines are peers in assessment, not subordinates to each other. The Orchestrator validates the canonical request, resolves minimum necessary context once, and routes a governed assessment to each mandatory engine. Each engine evaluates its own rules and returns structured findings with reason codes, obligations, policy references, evidence references, timestamps and expiry. The deterministic PDP then combines those findings using the approved precedence rules.

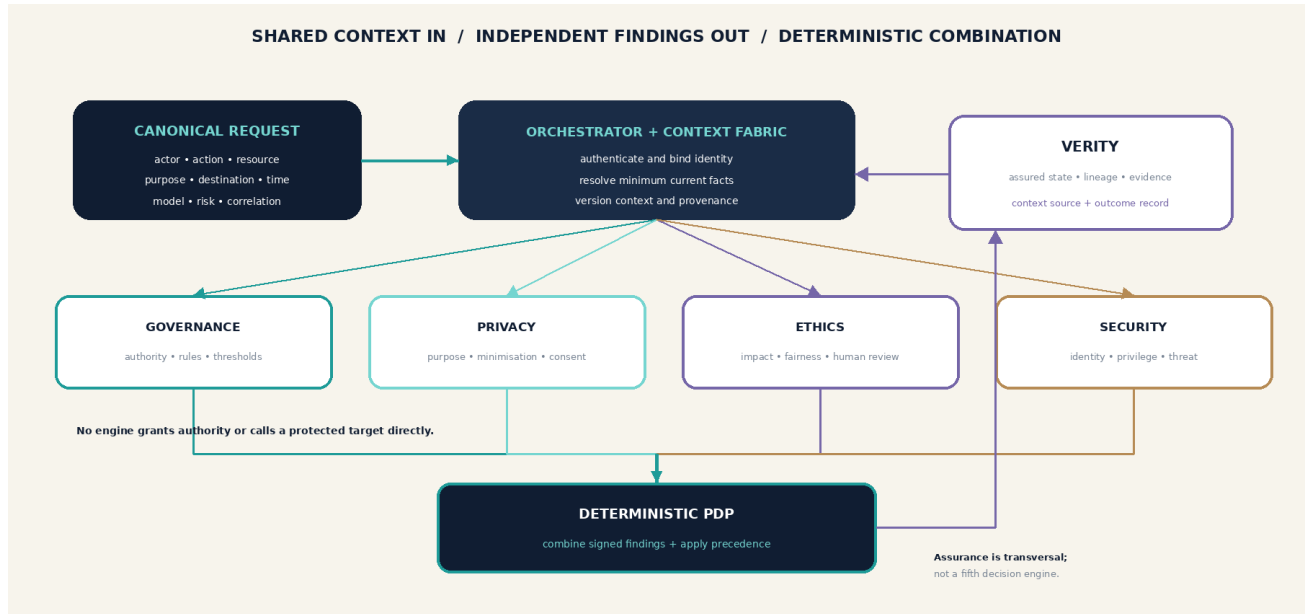


Figure 3. The four decision engines share a versioned request context; Verity supplies and records assurance evidence across the flow.

Engine	Primary question	Examples of evaluated context	How its result connects
Governance	Is this actor authorised to do this?	Delegated authority, business purpose, policy, thresholds, segregation of duties, ownership.	Can restrict scope or amount, require approval, or deny.
Privacy	May these data be used for this purpose?	Data class, purpose, consent, lawful-use configuration, processor, destination, minimisation, retention, residency.	Can exclude fields, require masking, limit use, or deny.
Ethics	Is the action acceptable and proportionate?	Consequence, affected groups, fairness concerns, vulnerability, transparency, human-oversight policy.	Can require explanation or human decision authority, escalate, or deny.
Security	Can this action be performed safely?	Actor/agent identity, authentication, privilege, tool, target, environment, threat and change context.	Can require stronger authentication, limit tool scope, escalate, or deny.

WHERE THE ENGINES CONNECT / **Coordination without hidden overrides**

Their connection is mediated through shared context, the Orchestrator and the PDP. The engines do not call one another to negotiate a result, and none can erase another engine’s mandatory hard denial. They can produce obligations that must be satisfied together: for example, privacy may require redaction while security requires multi-factor authentication and governance limits the permitted operation to a named customer case. The PDP preserves all applicable conditions; the PEP checks that each condition is enforceable before execution.

Verity participates across the system without becoming a fifth G/P/E/S engine. It can supply the state, lineage and assurance status of an asset to the context fabric; record the policy bundle and domain evidence behind the resulting decision; and issue or reference a manifest or receipt.

Datafolio provides the customer-controlled location for retained credentials and scoped presentations. Neither Verity nor Datafolio grants execution authority: that remains with the PDP and the PEP path.

If a domain times out or returns stale evidence, the Orchestrator marks the required result unresolved. The PDP applies the configured fail-closed or escalation policy. No success-shaped default is allowed. If the evidence status changes later, a ChangeSet and reassessment can suspend the prior assurance or require a new decision.

MODULAR COMPOSITION / Separate engines, shared operational contract

The modular model allows a customer to begin with the control domains required for a defined use case and to extend the configuration as its operating needs mature. Every enabled module still uses the shared action envelope, identity binding, reason-code conventions, decision transaction and assurance lifecycle. A module can add its specialist assessment and obligations; it cannot create a side door to the target system or issue execution authority.

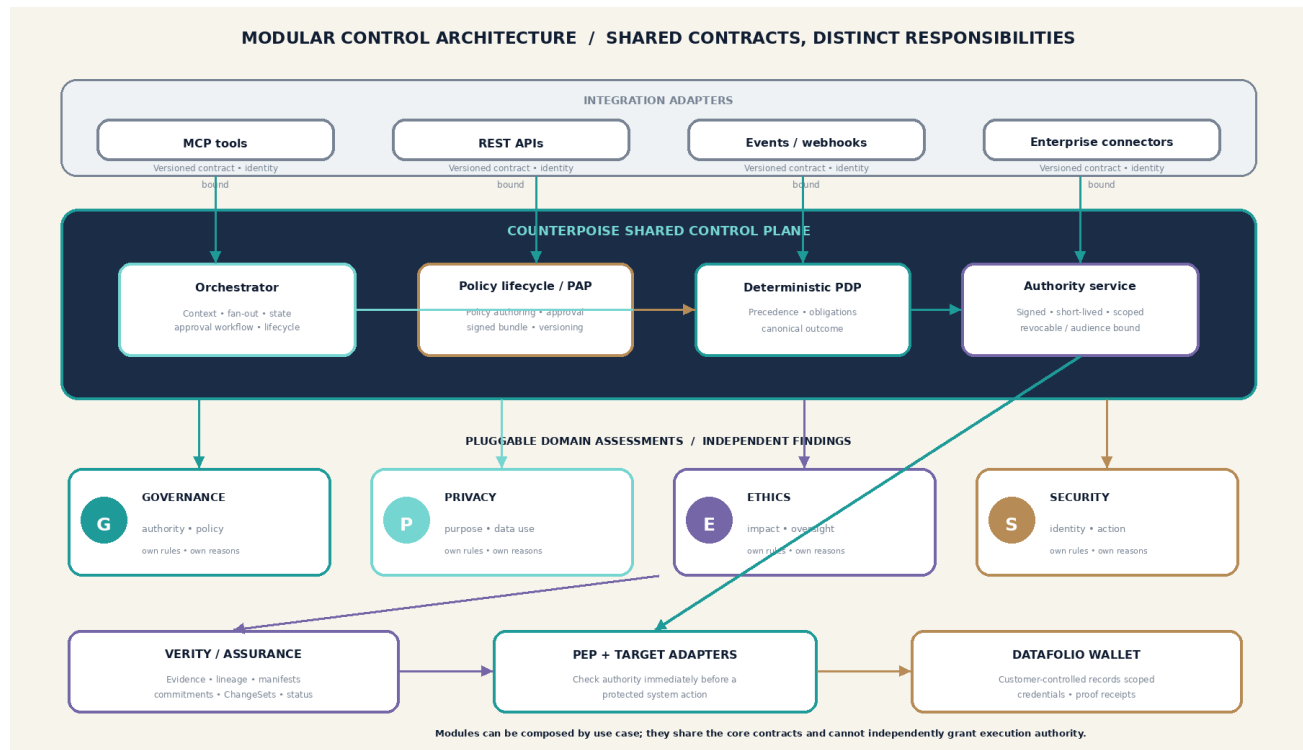


Figure 4. Modular architecture: distinct domain engines plug into shared control, enforcement and assurance services.

Verity is designed as a transversal capability because evidence and state assurance support more than one domain. For example, a data change can affect privacy purpose, governance approval and model risk at once. The affected modules are re-evaluated through Counterpoise using the changed asset version; Verity records the new assurance state. Datafolio retains a scoped credential or receipt when the customer chooses to use the wallet model.

03 / Decision lifecycle and outcome semantics

The lifecycle is a controlled transaction. It begins with proposed intent and ends with an execution outcome or an explicit non-execution state. The request, decision, approval, authority and execution event are linked by identifiers and trace context so a reviewer can reconstruct the chain later.

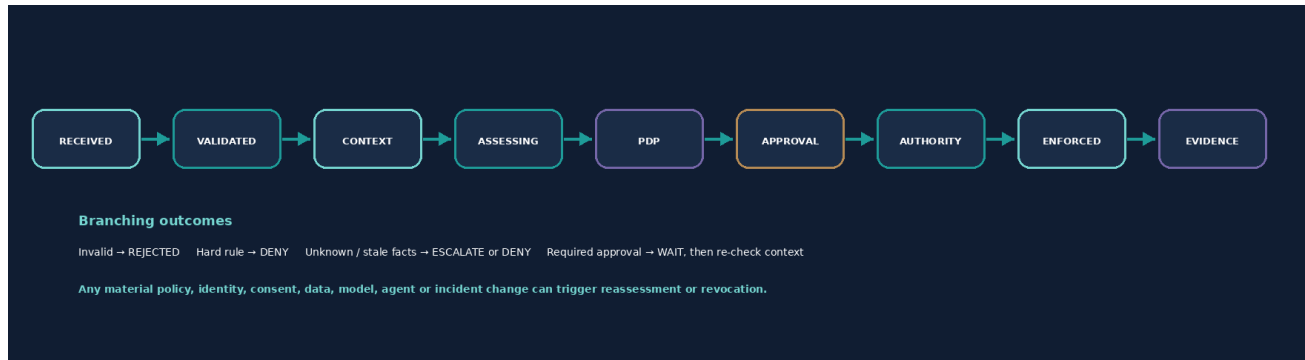


Figure 5. Typical decision states. Failure and approval branches do not issue execution authority.

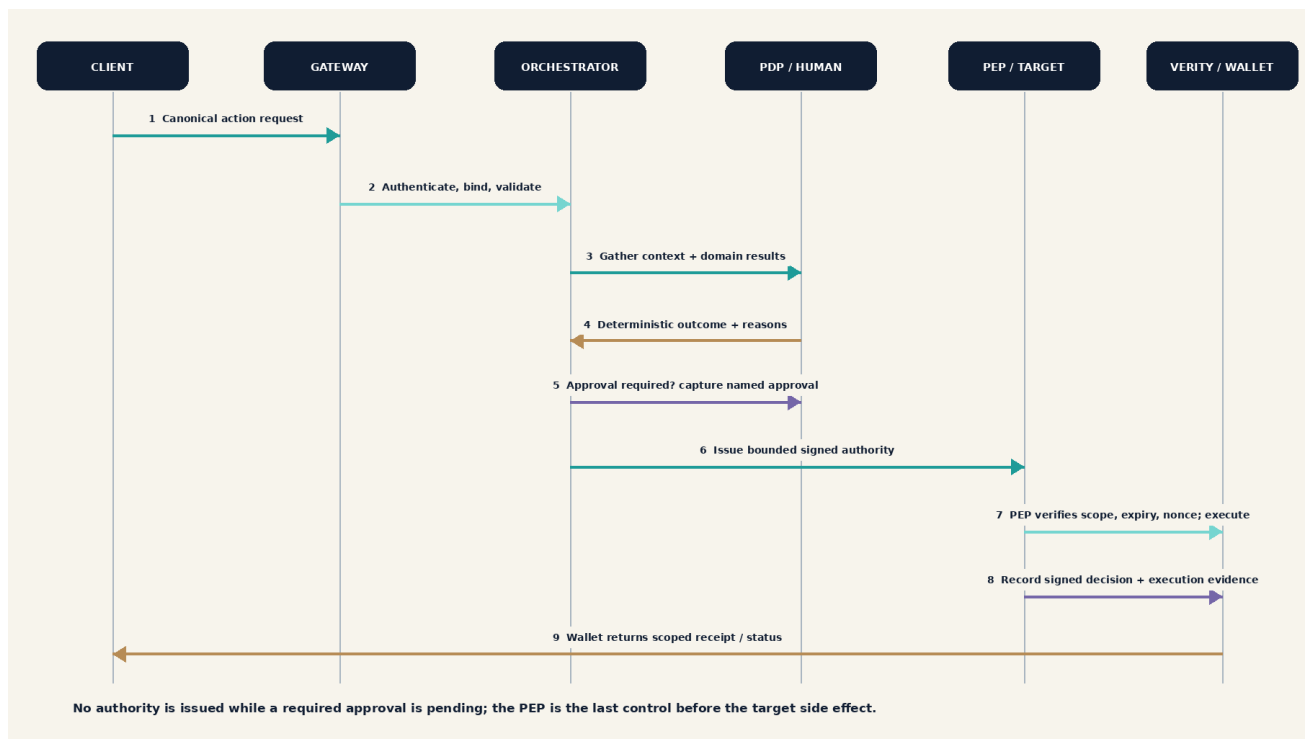


Figure 6. Sequence diagram for policy evaluation, execution enforcement and evidence return.

DECISION OUTCOMES / One canonical status for each result

Outcome	Meaning	Execution consequence
PERMIT	All mandatory controls have acceptable, current results.	PEP may execute only within the authority scope and lifetime.
PERMIT_WITH_CONDITIONS	Action may proceed only after enforceable obligations are applied.	PEP verifies or applies every obligation before execution.
REQUIRE_APPROVAL	Policy requires a named accountable person to decide.	No authority token until approval is captured, bound and context is rechecked.
ESCALATE	Context is insufficient, stale, conflicting or requires specialist review/redesign.	No execution; return the issue and its accountable owner.
DENY	A hard prohibition or unacceptable risk applies.	No execution; retain the decision and evidence.

COMBINING PRECEDENCE / How multiple findings resolve

A validated hard DENY from any mandatory domain wins. Unresolved mandatory facts lead to ESCALATE or DENY, according to risk policy. REQUIRE_APPROVAL is available only where policy explicitly allows accountable judgement. Unmet obligations prevent authority issuance. PERMIT is possible only when each mandatory domain has completed with an acceptable current result. Timeout behaviour is policy-specific, but consequential actions fail closed or escalate.

Approval is not a pause button detached from the request. The approval record must identify the approver, decision, action scope, conditions, rationale and expiry. Before resuming, the system rechecks material context. If the resource, policy, consent, identity or risk state has changed, the decision returns to evaluation instead of relying on an obsolete approval.

SIGNED AUTHORITY / Permission with a narrow envelope

If the PDP outcome permits execution, the authority service may issue a signed, short-lived token bound to the decision, actor, action, resource, target audience, policy version, nonce and expiry. The PEP verifies issuer, signature, audience, scope, obligations, decision state, expiry and replay protection immediately before the side effect. The target action is not called if any validation fails.

04 / Verity and the data assurance model

Verity is the assurance capability within Counterpoise. It binds an approved data or AI asset state to the context in which it was assessed. That state can include asset identity and version, selected attribute commitments, schema and canonicalisation profile, lineage references, validation

outcomes, policy version, Counterpoise decision, approvals, obligations, signatures and validity period.

The purpose is to answer practical questions: what version was assessed; which attributes or evidence were included; which rules and policy applied; who or what approved the state; what changed since then; and whether the currently presented state matches the recorded commitment. This can reduce ambiguity in downstream AI use and support review, but remains dependent on the quality of the source facts and governance process.

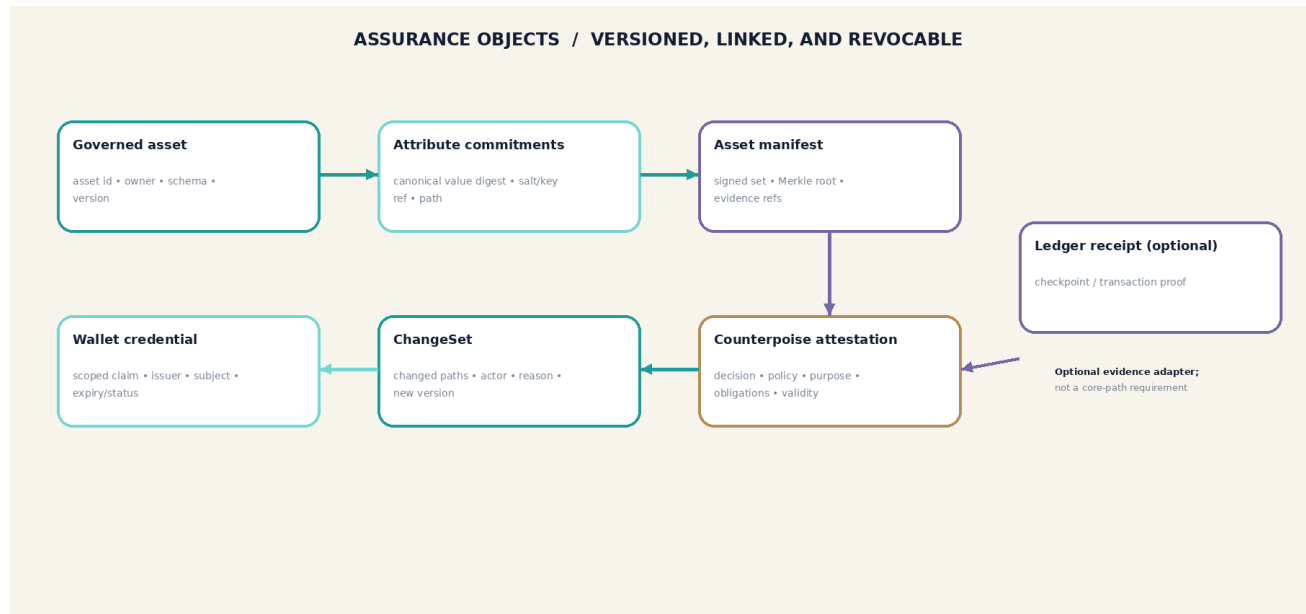


Figure 7. Core assurance objects and their relationships. A ledger receipt is an optional adapter.

INFORMATION MOVEMENT / Keep source records in their approved systems

The data path distinguishes an action request from the business payload on which a protected operation acts. Counterpoise receives the canonical envelope and minimum context needed to decide whether the operation may proceed. The business record remains in its approved source system by default. Only after a valid decision does the PEP allow the specific authorised operation to reach its target, subject to enforceable minimisation, masking, purpose, retention and destination conditions.

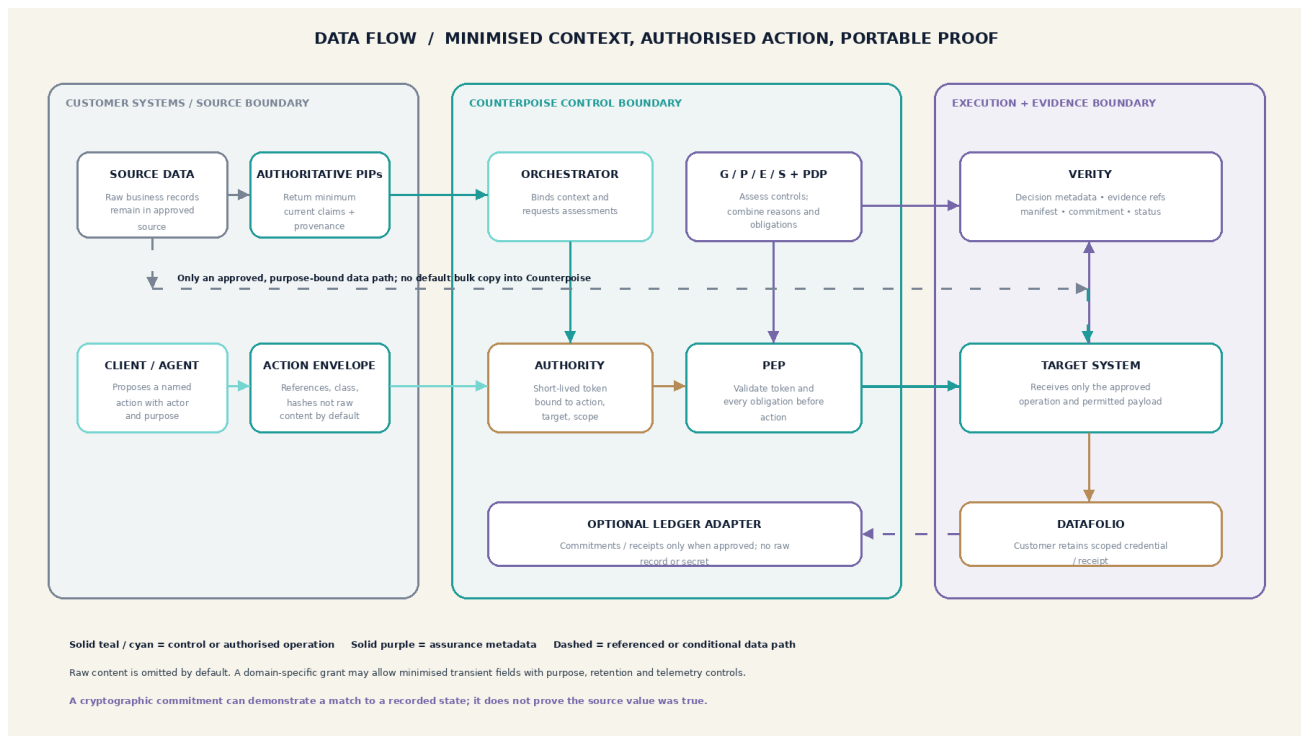


Figure 8. Data-flow view showing source boundary, control plane, authorised execution, assurance metadata and wallet receipt.

Verity records decision and evidence metadata, selected commitments and references rather than copying raw source records into the control plane by default. A narrowly scoped, transient content grant may be necessary for a particular assessment; that grant must specify fields, purpose, retention and telemetry suppression. Datafolio stores or references the customer's selected record, credential or receipt. An optional ledger can anchor a commitment or receipt after approval, but raw records, salts and keys must stay out of it.

COMMITMENTS AND MERKLE PROOFS / Tamper evidence without publishing raw data

A commitment can be generated over a canonical representation of a selected value and its context. In conceptual form: commitment = HASH(tenant reference || asset ID || version || attribute path || canonical value || salt || schema version). The canonicalisation profile and cryptographic algorithm must be versioned. Per-attribute random salt, or a keyed construction where threat modelling calls for it, helps reduce exposure to guessing attacks against low-entropy values. Salt and key material must remain protected and must not be written to a ledger.

A Merkle tree can combine multiple attribute commitments into a root digest. A holder can then present a selected attribute and its proof path to show that it belongs to the committed manifest without disclosing all other attributes. The verifier still needs to validate the issuer's signature, the relevant schema/profile, the credential's scope, status and expiry, and the policy context for which the proof is being used.

OFF-CHAIN AND OPTIONAL LEDGER BOUNDARY / **Keep sensitive evidence in approved systems**

Raw records, evidence files and sensitive source attributes remain in approved off-chain repositories or the customer's source system. A proof store may retain non-content commitments, signed manifests, timestamps, policy references and receipt identifiers, with access controlled to the assurance tier. A permissioned ledger is an optional adapter for cross-party verification; it is not a mandatory dependency in the consolidated core architecture. One reference design explores a permissioned Besu/QBFT network, but the authoritative baseline leaves consortium membership and vendor choice for separate approval.

Ledger unavailability must not create a false claim that a new commitment has been finalised. Asynchronous processing may be used only where policy allows it and the assurance state remains visibly pending until confirmation. Ledger records must avoid raw personal data and secrets; retention, erasure and data-protection impacts require review before any immutable identifier is committed.

ASSURANCE STATES / **State must be visible and reversible**

State	Interpretation	Typical handling
PENDING	Commit or external receipt has not reached its required confirmation point.	Do not present as fully assured; retry or investigate within policy.
ASSURED	Required checks and approvals have completed for the named version and scope.	May be relied on only within the stated purpose, expiry and policy conditions.
DRIFTED	The presented state differs from the recorded commitments.	Create a ChangeSet; restrict use or suspend dependent claims pending reassessment.
ESCALATED	A material change or evidence gap requires accountable review.	Route to the asset owner or designated specialist; do not silently renew assurance.
REVOKED / EXPIRED	The attestation or credential is no longer valid.	Reject use for controlled decisions; retain revocation/status evidence.

05 / Change detection and continuous reassessment

Assurance is versioned because data and operating context change. A source system, schema, consent record, model, policy bundle, agent identity or security condition may evolve after an assessment. Verity can compare a later presented state with prior commitments, map changed paths into a ChangeSet, and send the material change through Counterpoise for reassessment.

An authorised change creates a new version and a new signed manifest; it does not rewrite the prior evidence. An unauthorised or unresolved change moves the asset into a drifted, escalated or non-compliant status as defined by policy. Dependent attestations and wallet credentials can be suspended, expired or revoked. Events can notify the data owner, steward, auditor and integrated systems, but events themselves never grant execution authority.

- Asset change: a field, transformation, schema, lineage edge or source version changes.
- Policy change: the relevant policy bundle changes or a rule is withdrawn.
- Identity change: actor or agent credentials, autonomy level or delegated authority change.
- Consent or purpose change: permission to use information changes or expires.
- Model or agent change: version, configuration, tool set or operating context changes.
- Incident change: security signals or incident state require the previous decision to be re-evaluated.

The design should include status lookup, expected-version checks and idempotency for mutating operations. Conflict responses should make it clear when the asset version has already changed or a commitment has been superseded. Every change record should identify old and new versions, changed paths or commitments, initiating actor, reason and workflow state.

06 / Worked scenario

A PROCUREMENT AGENT PROPOSES A £75,000 PURCHASE / End-to-end control path

An autonomous procurement agent identifies a supplier and prepares a purchase request for £75,000. The organisation's policy permits the agent to prepare transactions but limits autonomous execution to £10,000. The action envelope identifies the agent, purchase operation, supplier and procurement resource, business purpose, destination finance system, relevant data classifications, model context and correlation identifiers. It does not carry invoice or customer content by default.

Stage	What happens	Control or evidence
1. Validate intent	Gateway authenticates the client and binds tenant/actor identity; schema and request shape are checked.	Rejected if identity, tenant binding or required fields are invalid.

Stage	What happens	Control or evidence
2. Resolve context	Orchestrator obtains current authority limits, supplier status, purchase policy, segregation-of-duties and risk facts.	Facts include provenance, version and timestamp.
3. Assess domains	Governance checks delegation and thresholds; Privacy checks data scope; Ethics checks relevant fairness/impact concerns; Security checks identity and destination.	Each mandatory engine returns structured findings and reason codes.
4. Combine	PDP applies the signed policy bundle. The £75,000 request exceeds the agent's autonomous threshold.	Outcome: ESCALATE or REQUIRE_APPROVAL, as configured.
5. Approve or revise	A named procurement approver reviews the action. If scope or facts change, the context is rechecked before decision resumes.	Approval bound to the amount, supplier, purpose, resource and expiry.
6. Enforce	If the final decision permits a bounded action, a signed authority is issued. PEP validates it directly before calling finance.	No authority or no side effect while approval is pending or denied.
7. Record	Verity records decision context, policy version, approval, token validation and execution outcome; a receipt can be retained in Datafolio.	Evidence is linked and integrity protected; raw financial records stay in the finance system.

The same pattern applies to a production firewall change: an operations agent may inspect and propose, while the PEP blocks a mutating tool call unless an authorised security engineer approves it and a policy-bound token permits that exact action. In a high-impact employment workflow, an ethics rule can require human decision authority even when other checks pass.

07 / Interfaces and information contracts

Clients integrate through a public MCP gateway, REST APIs or event contracts, according to the customer's integration pattern. AI clients receive only approved business-level tools. Internal engines may use MCP, REST or gRPC adapters, but are not directly exposed to agents as a route around the control plane. Read-only assessment tools and high-control mutating tools must be distinguished in the catalogue.

CANONICAL ACTION ENVELOPE / Minimum context to evaluate intent

A versioned action envelope includes a request ID, tenant reference, actor type and identity, client and authentication context, action name and operation, target resource, purpose, destination, jurisdiction, resource classifications or references, model/agent version and autonomy context, declared risk, policy-set identifier, requested timestamp and trace/correlation identifiers. Raw content is omitted by default. A domain-specific transient content grant may provide minimised fields with explicit purpose, retention and telemetry-suppression controls.

STRUCTURED DOMAIN RESULTS / Explainable machine-readable findings

Each domain result includes its domain, assessment and decision identifiers, status, reason codes, obligations, policy references, evidence references, assessment time, expiry, result digest and signature or integrity protection. The final decision references the domain results, policy bundle, obligations and any bounded authority. Human-readable explanation supplements these fields but does not replace them.

EXAMPLE DECISION ENVELOPE / Illustrative JSON

```
{
  "decision_id": "DEC-01J...",
  "request_id": "REQ-2026-009821",
  "outcome": "PERMIT_WITH_CONDITIONS",
  "reason_codes": ["PRIVACY.MINIMISE", "SECURITY.MFA_REQUIRED"],
  "obligations": [
    {"type": "redact", "fields": ["customer.dob"]}
  ],
  "policy_bundle": "gopes-core@1.0",
  "authority": {
    "audience": "customer-crm-pep",
    "scope": ["customer.create_case"],
    "resource_hash": "sha256:...",
    "expires_at": "RFC3339 timestamp",
    "nonce": "unique-value",
    "signature": "..."
  },
  "evidence_ref": "wallet://assurance/..."
}
```

COMPATIBILITY / Contracts should travel across deployment profiles

Schemas use semantic versioning; incompatible changes require a new major interface version. JSON canonicalisation and signing algorithms are versioned. Mutating endpoints use idempotency keys and expected-version protection where concurrent updates can conflict. The Azure and Heroku profiles must implement the same schema versions, outcome taxonomy, reason-code catalogue and golden-scenario semantics.

08 / Security, privacy and operational assurance

A policy control plane is itself security-sensitive. It can make or block consequential actions, so its trust boundaries and failure behaviour require explicit design. The implementation must threat-model agent impersonation, tenant confusion, token replay, stale approvals, excessive scope, PEP bypass, compromised signing keys, evidence tampering, malicious tool calls and dependency outages.

Control area	Technical requirement
Identity and tenancy	Bind tenant and actor to authenticated claims; verify issuer, audience and scopes; preserve the human actor where delegated access is used.
Authority	Use short validity, explicit audience/resource/action scope, nonce or equivalent replay defence, signature verification and revocation/status checks.
Enforcement	Put PEP checks on every protected side effect; test that direct calls cannot bypass the PEP.
Policy lifecycle	Review, test, sign and stage policy bundles; support shadow evaluation, staged promotion and rollback.
Cryptographic keys	Use managed signing keys with least privilege, rotation and compromise runbooks; keep salts and secrets out of manifests and ledgers.
Privacy minimisation	Omit raw content by default; use narrow references or transient grants; suppress customer-content telemetry and define retention.
Evidence integrity	Use append-only or immutable evidence controls appropriate to assurance tier; link records with signatures, digests and provenance.
Resilience	Test fail-closed or escalation paths for PDP, PIP, approval, event, signing, evidence and target-system failures.
Observability	Emit correlated traces, metrics and audit events while avoiding customer-content logging; alert on denial spikes and possible PEP bypass.
Separation of duties	Separate policy authoring, policy approval, platform release, exception approval, execution and evidence custody.

FAILURE SEMANTICS / When dependencies do not respond

Failure behaviour is risk-specific but must be explicit. A missing mandatory result, stale fact, expired approval or invalid signature cannot silently become PERMIT. High-consequence actions should fail closed or escalate. If a ledger adapter or evidence store is unavailable, the system must not report a commitment as final; pending status, retry and recovery actions must be visible. Operational runbooks should cover signing-key compromise, approval backlog, PIP or queue outage, policy rollback, tenant-isolation incident, evidence degradation and PEP bypass detection.

09 / Deployment profiles and portability

The product is defined by a provider-neutral logical architecture. Azure and Heroku are deployment profiles of that architecture, not separate products with different authority semantics. Azure is the strategic enterprise profile; Heroku is intended for lower operational overhead in SME or controlled-exception contexts. Profile selection depends on customer requirements, data sensitivity, integrations, scale, resilience and operational ownership.

Invariant	Must remain consistent
Contracts	Schema versions, REST/MCP/event semantics, identifiers and compatibility rules.
Decision model	Canonical outcomes, combining precedence, reason-code meanings and policy interpretation.
Authority	Token scope, signing rules, expiry, audience binding, replay resistance, revocation and PEP validation.
Assurance	Manifest meaning, evidence linkage, data boundary, status semantics and wallet references.
Conformance	The same golden inputs and signed policy digest produce semantically equivalent results.
Trust separation	Development, test and production use separate trust boundaries and keys; images are signed and promoted by digest.
Observability	Compatible trace, metric and audit event structure without customer content in telemetry by default.

Provider-specific databases, queues, key services, identity providers and deployment services sit behind adapters. Runtime configuration may differ, but business outcomes and enforcement rules may not. An optional ledger adapter can be enabled for a customer assurance tier without making public-chain participation or ledger availability part of every action's critical path.

10 / Engineering, testing and production readiness

A safe implementation starts by freezing the canonical contracts: action envelope, domain result, decision response, outcome semantics, reason-code namespace, evidence manifest and signing canonicalisation. Client and server types can then be generated from schemas and validated through contract tests. A change to a normative rule or trust boundary should follow an Architecture Decision Record, impact assessment, security/privacy review, compatibility analysis and update to golden scenarios.

MINIMUM VALIDATION GATES / Evidence required before controlled release

- Unit and policy tests for rules, obligations, precedence, expiry and revocation.
- Golden scenarios covering PERMIT, conditional permit, approval, escalation, denial, timeout and stale context.
- Contract tests for REST, MCP, event and schema compatibility.
- Adversarial tests for replay, cross-tenant reuse, privilege escalation, prompt injection paths and target bypass.

- Cross-profile conformance tests proving Azure and Heroku semantic equivalence.
- Integration tests that reconstruct a full decision, approval, token validation, PEP execution and wallet receipt.
- Recovery tests for key rotation, policy rollback, evidence degradation, queue failure and service outages.
- Release evidence including software bill of materials, vulnerability results, licensing evidence, signed build provenance and rollback plan.

Production readiness also includes named control owners, threat model, privacy impact review, data inventory, service objectives, monitoring and alert thresholds, incident procedures, backup and recovery, support model and user-facing privacy notices. The assurance tier should determine the evidence retention, immutability and independent verification requirements.

11 / Use cases across the control domains

Domain	Question at decision time	Illustrative control
Governance	Is this actor authorised to take this action, for this amount or consequence?	Procurement agent prepares a £75,000 purchase; the £10,000 autonomous threshold triggers escalation.
Privacy	May this information be used for this purpose, by this processor and destination?	Customer-retention workflow masks or excludes special-category attributes and applies retention obligations.
Ethics	Is the action acceptable, proportionate and subject to human decision authority?	Employment ranking can inform analysis but final redundancy decisions remain with authorised people.
Security	Can this agent or user safely perform the requested operation with this tool and privilege?	Infrastructure agent may inspect a firewall but cannot change production rules without approved authority.
Data assurance	Is the present asset state the same version that was assessed and approved?	Changed upstream attributes create a ChangeSet, trigger reassessment and can suspend the prior attestation.

The cross-domain pattern is more important than any one example. A request can be permitted for analysis but denied for autonomous execution; allowed with redaction; paused for named approval; or blocked when an evidence state has drifted. The platform should express those outcomes as machine-readable obligations and authority boundaries, not only as prose warnings to the agent.

12 / **Claims, limitations and responsible positioning**

The architecture is intended to make policy decisions more executable and evidence more reviewable. It does not remove the need for good policy, lawful processing, accurate source systems, competent human judgement, independent security controls or regulatory interpretation. A signed decision record proves what the system recorded and under which context; a cryptographic commitment can show whether a presented value matches a committed value. Neither establishes the real-world truth of the original source claim.

Counterpoise, Verity and Datafolio are described here as product capabilities in development. The paper explains intended architecture and design requirements; it is not evidence that every feature is deployed, independently tested or operationally validated. It does not represent a certification, guarantee of compliance, guarantee of data truth, absence of model error or elimination of cyber risk. Customer-specific policies, integration contracts, assurance tier, wallet custody and any ledger design require separate validation and approval.

RECOMMENDED INTERPRETATION / What the architecture can reasonably support

When implemented and validated for a defined use case, the control loop can support consistent action-level policy evaluation, explicit authority boundaries, human approval capture, enforcement checks and provenance-linked evidence. The organisation remains accountable for selecting sound policies, maintaining source-data quality, defining legitimate purposes, operating secure systems and deciding what level of evidence is sufficient.

CONCLUSION / Useful autonomy requires explicit authority

AI adoption becomes more consequential when systems can act across data, tools and business processes. An organisation needs a way to identify the actor, understand the requested operation, test the purpose and context, impose the right conditions, pause for a human where required, block prohibited actions and reconstruct what happened afterwards.

Counterpoise provides the proposed control plane for that decision. Verity links assurance evidence to the data and policy state on which decisions depend. Datafolio gives customers a wallet-first way to retain and present scoped records and proof. Together, the design aims to make responsible delegation operational: authority is specific, decisions are deterministic, evidence is traceable and people remain accountable.

APPENDIX A / Source basis and terminology

This white paper is a reader-oriented technical synthesis of the AIdentity design set. The consolidated master architecture and implementation specification dated 15 September 2026 is the controlling baseline for canonical contracts, decision semantics, provider portability and wallet-first boundaries. Detailed Verity material is used to explain assurance objects and optional ledger patterns; where earlier reference material proposes a specific vendor or ledger, it is treated as an option unless the master baseline makes it normative.

Source	Use in this paper
AIdentity Counterpoise Master Architecture and Implementation Specification, v1.0, 15 September 2026	Controlling architecture principles, canonical outcomes, trust boundaries, wallet-first data handling, Azure/Heroku invariants, test and release requirements.
GoPES Orchestrator Technical Architecture White Paper, v1.0, 15 September 2026	Orchestration lifecycle, state model, human approval, PDP/PEP separation and event flow.
GoPES Data Assurance Layer / Verity technical design material, September 2026	Assurance manifests, attribute commitments, Merkle proofs, ChangeSets, wallet credentials, ledger receipts and change-state handling.
Counterpoise 2.0 Integrated Architecture UML Specification, v1.0, 11 September 2026	UML-aligned component concepts, decision record and integration architecture.
Datalogos Limited Branding Playbook, September 2026	Brand architecture, typography, palette and precise product naming.

TERMINOLOGY / Abbreviations used

Term	Meaning
PAP	Policy Administration Point: manages approved policy and policy bundles.
PDP	Policy Decision Point: deterministically combines required findings into the canonical outcome.
PEP	Policy Enforcement Point: validates decision authority before the protected side effect.
PIP	Policy Information Point: supplies authoritative facts required for policy evaluation.
MCP	Model Context Protocol: one supported protocol surface for approved client-to-platform tools.
DAL	Data Assurance Layer: assurance function referred to as Verity in the current portfolio naming.
Manifest	Signed versioned record binding selected asset state, commitments and evidence references.
ChangeSet	Structured description of differences between asset versions, with actor, reason and workflow state.